深度学习基础

第 7 讲 — 经典 CNN 架构与设计思想

孟翰

人工智能学院
中国石油大学（北京）

2026 年春季学期

本讲内容

1. 回顾与本讲目标
2. AlexNet：深度学习时代的开启
3. VGG：堆叠小核的极致美学
4. GoogLeNet：Inception 模块与多尺度
5. ResNet：残差连接与超深网络
6. 设计思想总结与现代 CNN

核心抽象

卷积 = 局部连接 + 参数共享 ； **池化** = 局部聚合 + 平移鲁棒

LeNet 范式（1998）

Input → Conv → Pool → Conv → Pool → FC → FC → Output

下一个大问题

LeNet 在 MNIST 上很好用，但放到**真实大图（ImageNet** 224×224**，1000 类）**就不行了。需要怎样的改进？这就是 2012-2015 年间发生的事情。

一句话目标

理解 **LeNet** → **AlexNet** → **VGG** → **GoogLeNet** → **ResNet** 每一步**解决的问题**和**设计思想**。

时间线

年份	网络	核心贡献
1998	LeNet	卷积 + 池化范式
2012	**AlexNet**	ImageNet 冠军，深度学习时代开启
2014	VGG	堆叠小核做深
2014	GoogLeNet	Inception 模块、多尺度并行
2015	**ResNet**	残差连接，深度突破 1000 层

学完本讲，你能

读懂视觉论文中的 “backbone” 描述；理解为什么 “深” 比 “宽” 重要；会自己设计 CNN。

三位作者（多伦多大学）

- **Alex Krizhevsky**（名字由来）：Hinton 博士生，**一个人在宿舍用两块 GTX 580** 训练出 AlexNet。
- **Ilya Sutskever**：同为 Hinton 学生，后来成为 **OpenAI 联合创始人兼首席科学家**，ChatGPT 之父之一。
- **Geoffrey Hinton**：**"深度学习之父"**。2018 年图灵奖得主，**2024 年诺贝尔物理学奖**得主。

关键八卦

2012 年夺冠后，三人公司 DNNresearch 被 **Google 以 4400 万美元收购**。Hinton 加入 Google，**深度学习正式进入工业界**。

论文信息

ImageNet Classification with Deep CNN
NeurIPS 2012

引用 > **17 万次**

小故事

当时 Hinton 在学术界几乎被边缘化，这篇论文让整个 CV 社区**一夜之间**转向深度学习。

ImageNet（2009-）

1400 万标注图像，2 万多类别。ILSVRC 竞赛子集：1000 类，120 万训练。由李飞飞团队构建。

2012：深度学习元年

AlexNet 把 top-5 错误率从 25.8% 降到 16.4%，**绝对差距 10 个百分点**。

ILSVRC 冠军 Top-5 错误率

年份	方法	Top-5
2010	传统 CV (SIFT+SVM)	28.2%
2011	传统 CV (改进)	25.8%
2012	**AlexNet**	**16.4%**
2014	VGG	7.3%
2015	ResNet	**3.6%**

为什么 2012 是奇点

大数据（ImageNet）+ **大算力**（GPU）+ **大模型**（CNN+ReLU+Dropout）三者首次同时具备。

基本配置

层	操作	输出	说明
Input	—	$3\times224\times224$	RGB
Conv1	11×11, 96, $s=4$	$96\times55\times55$	大核大步幅
Pool1	MaxPool 3×3, $s=2$	$96\times27\times27$	
Conv2	5×5, 256, $p=2$	$256\times27\times27$	
Pool2	MaxPool 3×3, $s=2$	$256\times13\times13$	
Conv3,4,5	3×3, 384/384/256	$256\times13\times13$	三层堆叠
Pool5	MaxPool 3×3, $s=2$	$256\times6\times6$	
FC6	4096	4096	+ Dropout
FC7	4096	4096	+ Dropout
FC8	1000	1000	Softmax
总参数		~ 6000 万	

1. ReLU 取代 Sigmoid/Tanh

解决深层网络梯度消失，训练速度快 6 倍。**今天依然是默认激活。**

2. Dropout 防止过拟合

全连接层用 $p = 0.5$ 的 Dropout，让模型更鲁棒。

3. 数据增强 + 4. 双 GPU 并行

随机裁剪 + 水平翻转 + 颜色扰动 $\Rightarrow$ 训练集扩大数千倍。单 GPU 显存不够（3GB），把网络切成两半在两块 GPU 上跑。

5. LRN 局部响应归一化（已被淘汰）

后来被 BatchNorm 取代。**这是少数被历史否定的设计。**

问题 1：大核 + 大步幅丢信息

11×11 卷积核 + stride=4，第一层就把图像缩小 4 倍。**细节信息一开始就丢掉了。**

问题 2：架构不规整

卷积核大小 $11 \to 5 \to 3 \to 3 \to 3$，没有统一规律，难以扩展和复制。

问题 3：参数主要在 FC 层

6000 万参数中，~ 5800 **万在 3 个 FC 层**。这些参数大多冗余。

下一步：VGG

VGG 团队提出：**用极简、极规整的设计——只用 3×3 卷积，只用 stride=1，只用 2×2 池化。**

Visual Geometry Group (VGG)

VGG 不是人名，而是**牛津大学 Visual Geometry Group** 的缩写。论文两位一作：

- **Karen Simonyan**：后加入 **DeepMind**，参与 **AlphaGo / AlphaZero**；现为 **Microsoft AI 首席科学家**。
- **Andrew Zisserman**：**CV 泰斗**，VGG 组创始人。教科书 *Multiple View Geometry* 是 SLAM/三维重建圣经。

论文定位

发表在 **ICLR 2015**，为参加 2014 ILSVRC。**亚军**（输给 GoogLeNet），但因架构极简，后世使用率反而更高。

机构

University of Oxford

Visual Geometry Group

“简洁的胜利”

GoogLeNet 精心设计、多辅助分类器；VGG 是 **“蛮力美学”**——只堆 3×3。**简洁往往比精巧更容易被后人复用。**

设计哲学

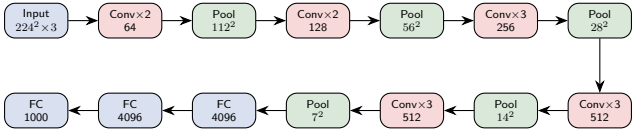
所有卷积层都用 3×3**，所有池化都用** 2×2。其他什么都不做。

为什么 3×3？三个洞察

1. **两个** 3×3 = **一个** 5×5 **感受野**（用第 6 讲公式可验证）
2. **三个** 3×3 = **一个** 7×7 **感受野**
3. 但是参数更少、非线性更多

量化对比（输入通道 = 输出通道 = C）

	单 7×7	三 3×3	节省
参数量	$49C^2$	$27C^2$	**45%**
非线性层数	1	**3**	



核心规律

每经过一次池化，空间减半、通道翻倍（64→128→256→512）。这是**特征金字塔**思想的明确体现。

VGG 的代价

1.38 亿参数（比 AlexNet 多一倍多），训练几周。原因：FC 层 $7 \times 7 \times 512 \times 4096 \approx 1$ 亿参数。

```python
def vgg_block(num_convs, in_channels, out_channels):
    layers = []
    for _ in range(num_convs):
        layers.append(nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1))
        layers.append(nn.ReLU(inplace=True))
        in_channels = out_channels
    layers.append(nn.MaxPool2d(kernel_size=2, stride=2))
    return nn.Sequential(*layers)

# VGG-11 配置：(num_convs, out_channels)
conv_arch = [(1, 64), (1, 128), (2, 256), (2, 512), (2, 512)]

class VGG(nn.Module):
    def __init__(self):
        super().__init__()
        in_c, blocks = 3, []
        for n, out_c in conv_arch:
            blocks.append(vgg_block(n, in_c, out_c))
            in_c = out_c
        self.features = nn.Sequential(*blocks)
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(512*7*7, 4096), nn.ReLU(), nn.Dropout(0.5),
            nn.Linear(4096, 4096),    nn.ReLU(), nn.Dropout(0.5),
            nn.Linear(4096, 1000),
        )
```

VGG 留给后世的关键思想

1. **用“块”组织网络**：vgg_block 可以反复堆叠
2. **统一卷积大小**：以后的网络几乎都用 3×3
3. **深度的力量**：从 AlexNet 的 8 层到 VGG 的 19 层，精度持续提升

VGG 的局限

- **参数太多**：FC 层占大头
- **无法做得更深**：超过 20 层后梯度消失重新出现
- **计算量大**：训练慢

下一步

有没有办法**既深又轻**？这是 GoogLeNet 和 ResNet 分别给出的两个答案。

一作与核心贡献者

- **Christian Szegedy**（一作）：Google 研究员，**对抗样本**概念提出者，深度学习安全领域开创者。
- **Wei Liu**：后提出目标检测里的 **SSD**。
- **Yangqing Jia（贾扬清）**：**Caffe 框架作者**，Facebook AI 创始成员，现为 Lepton AI CEO。

命名的小心思

"Google**L**eNet" 的 **L** 致敬 **LeCun 的 LeNet**。"Inception" 来自电影盗梦空间台词：**"We need to go deeper"**。

机构

Google Research

CVPR 2015

比赛结果

2014 年 ILSVRC **冠军**（VGG 亚军）。Google 由此**确立话语权**，后续 Inception-v2/v3/v4 / Xception 都源自这一脉。

动机 1：多尺度问题（最核心）

一张图里物体大小差异巨大：**小鸟** $\sim$ 30 像素，**房子** $\sim$ 150 像素，**天空**占整张图。传统 CNN 每层只用**一种**卷积核，等于 “用同一把尺子量所有东西”。

方案：同一层并行用 1×1 / 3×3 / 5×5 多种核**同时捕获多尺度特征**（这正是 “Inception” 之名由来）。

动机 2：选择困难症

每层到底用哪种核？要不要池化？调参成本巨大。**方案：都用！**反向传播自己学每个分支的重要性。

动机 3：参数效率

VGG 有 1.38 亿参数，绝大多数浪费在 FC 层。**方案**：1×1 卷积降维 + GAP 取代 FC。

结果

GoogLeNet（22 层）参数仅 **700 万**，约 VGG 的 $1/20$，精度还更高。

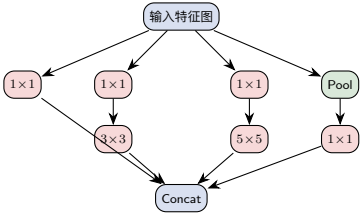
Inception v1 结构

四个并行分支，输出在通道维拼接：

- 分支 1：1×1 卷积
- 分支 2：$1\times1+3\times3$
- 分支 3：$1\times1+5\times5$
- 分支 4：MaxPool $+\ 1\times1$

1×1 卷积的关键作用

降维：把通道从大变小，减少后续 $3\times3/5\times5$ 的计算量。



1×1 卷积做了什么？

对每个空间位置，将 C_{in} 维的通道向量**线性变换**到 C_{out} 维。**它本质上是逐位置的全连接。**

三大用途

1. **降维**（Inception 中的核心用法）：$256 \to 64$
2. **升维**（ResNet bottleneck 中）：$64 \to 256$
3. **增加非线性**（配合 ReLU）

量化降维收益

Inception 中先用 1×1 把 256 通道压到 64，再做 5×5 卷积：

$$\text{原}: 256\times256\times5^2 = 1.6\text{M} \to \text{新}: 256\times64+64\times64\times5^2 = 120\text{K}$$

计算量减少 13 倍。

传统做法（VGG）

最后一层卷积输出 $7\times7\times512$
$\rightarrow$ Flatten $\rightarrow$ Linear$(25088, 4096)$
$\rightarrow$ Linear$(4096, 1000)$
~ 1 **亿参数**

GAP 做法（GoogLeNet）

最后一层卷积输出 $7\times7\times1024$
$\rightarrow$ AvgPool 到 $1\times1\times1024$
$\rightarrow$ Linear$(1024, 1000)$
~ 100 **万参数**

为什么 GAP 有效

- 每个通道 $\rightarrow$ 一个数字 $\rightarrow$ 一个 “概念”
- 这种约束**逼迫卷积层学习语义特征**
- **无参数** $\rightarrow$ 不易过拟合
- **对输入尺寸鲁棒**

影响

GAP 从此成为现代 CNN 的**标准结尾**（ResNet/DenseNet/EfficientNet 都用）。

四位作者，全是华人

- **何恺明（Kaiming He）**：一作，**当代最重要的 CV 研究者之一**。清华本科、港中大博士。后去 **FAIR**，又提出 Mask R-CNN、MoCo、MAE。2024 年回 **MIT** 任教授。
- **张祥雨**：原**旷视研究院**负责人，ShuffleNet 作者。
- **任少卿**：**Faster R-CNN** 一作，现为**蔚来自动驾驶副总裁**。
- **孙剑 (1976–2022)** †：时任 MSRA 首席研究员，后任**旷视首席科学家**。**2022 年 6 月因突发疾病去世，享年 45 岁**。中国 CV 界的重大损失。

机构

Microsoft Research Asia (MSRA)
北京中关村

2015 战绩：一篇论文五项冠军

ILSVRC 分类/检测/定位 + COCO 检测/分割。

CVPR 2016 最佳论文奖。引用 > 23 万次（**CS 史上引用最高论文之一**）。

反直觉的现象（2015 年前）

更深的网络应该更强。但实验发现：**20 层的网络比 56 层的网络更准**。为什么？

排查

- 不是过拟合：训练误差也更高
- 不是梯度消失：BatchNorm 已经解决
- 是**优化困难**：深层网络很难训练

ResNet 的洞察（何恺明等，2015）

让网络学习**残差** $F(x) = H(x) - x$，而不是直接学习目标 $H(x)$。
如果最优是恒等映射 $H(x) = x$，让 $F(x) = 0$ 比让 $H(x) = x$ 容易得多。

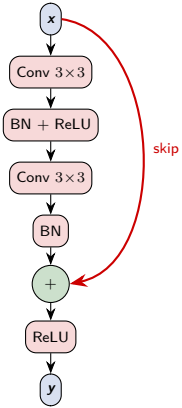
核心公式

$$\boldsymbol{y} = F(\boldsymbol{x}, \{W_i\}) + \boldsymbol{x}$$

Skip Connection（跳跃连接）：把输入**直接加到输出**。

为什么有效

- 梯度可以**直通**回去
- 学习恒等映射不再需要“凑”
- **加深不再损害精度**



ResNet 家族

	ResNet-18	ResNet-34	ResNet-50	ResNet-152
层数	18	34	50	152
块类型	基本块	基本块	Bottleneck	Bottleneck
参数量	1100 万	2150 万	2560 万	6020 万
ImageNet Top-1	70.0%	73.3%	76.0%	77.8%

Bottleneck 块（用于深层）

为了控制计算量，深层用 $1\times1 \to 3\times3 \to 1\times1$ 三层结构，先降通道再升回来。

ResNet 的真正意义

它让“加深网络”成为安全操作。从此 152、200、1000 层都成为可能。后续大部分视觉论文都以 ResNet 为 backbone。

```python
class BasicBlock(nn.Module):
    def __init__(self, in_c, out_c, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_c, out_c, 3, stride=stride, padding=1, bias=False)
        self.bn1   = nn.BatchNorm2d(out_c)
        self.conv2 = nn.Conv2d(out_c, out_c, 3, stride=1, padding=1, bias=False)
        self.bn2   = nn.BatchNorm2d(out_c)
        # 当通道数或空间尺寸不一致时，需要1x1卷积调整x
        self.shortcut = nn.Identity()
        if stride != 1 or in_c != out_c:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_c, out_c, 1, stride=stride, bias=False),
                nn.BatchNorm2d(out_c),
            )

    def forward(self, x):
        out = torch.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out = out + self.shortcut(x)   # <-- 关键：残差连接
        return torch.relu(out)
```

解释 1：梯度高速公路

反向传播时，梯度可以**直接**通过 skip connection 流回浅层，避免多次连乘导致的衰减。

解释 2：集成学习视角

N 层 ResNet 可以视为 2^N 个不同深度子网络的**隐式集成**（每个块可以 “跳过” 或 “经过”）。

解释 3：Loss landscape 平滑

可视化研究表明，残差连接使 loss landscape 显著**更平滑**，更容易找到好极小值。

核心结论

残差连接是**深度学习历史上最重要的架构创新之一**。今天几乎所有深网络都用它（包括 Transformer！）。

主线 1：让网络更深

LeNet (5) → AlexNet (8) → VGG (19) → GoogLeNet (22) → ResNet (152) → ResNet (1000+)
深度的限制被残差连接彻底打破。

主线 2：让参数更少

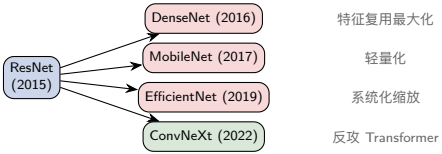
AlexNet (60M) → VGG (138M) → GoogLeNet (7M) → ResNet-50 (25M)
靠 GAP 取消 FC 层 + Bottleneck 降维。

主线 3：模块化设计

LeNet/AlexNet 是**手工堆叠** → VGG 是 **vgg_block** → GoogLeNet 是 **Inception** → ResNet 是 **Residual Block**。
以 "块" 为单位思考是现代架构的核心方法。

ResNet 之后还发生了什么？四个新问题

- **特征能不能更好复用？ → DenseNet**
- **能不能在手机上跑？ → MobileNet**
- **如何系统地“缩放”网络？ → EfficientNet**
- **CNN 真的被 Transformer 淘汰了吗？ → ConvNeXt**

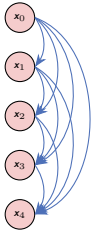


核心思想

每一层都连接到后面的所有层（不只是相邻层）。信息和梯度都能**直接**流到任意位置。

对比 ResNet

- ResNet：$\boldsymbol{y}_l = F(\boldsymbol{y}_{l-1}) + \boldsymbol{y}_{l-1}$ **（相加）**
- DenseNet：$\boldsymbol{y}_l = F([\boldsymbol{y}_0, \boldsymbol{y}_1, \ldots, \boldsymbol{y}_{l-1}])$ **（拼接）**



特点与代价

优点：参数更少（约 ResNet 的 40%），梯度流动最顺畅，特征复用极致。

缺点：**显存占用高**（要保存所有中间特征），工程不便。

作者与荣誉

Gao Huang（清华）& Kilian Weinberger 等。

CVPR 2017 最佳论文。

标准 3×3 卷积的“两件事”（往往被忽略）

一个标准的 3×3 卷积层，**同时**在做两件事：

1. **空间滤波**：在每个通道的 3×3 邻域内提取**空间模式**（边缘/纹理）
2. **通道融合**：把所有 C_{in} 个输入通道的信息**线性组合**起来

数学上

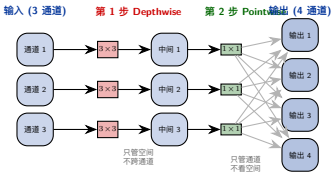
每个输出值 $=\sum_{c=1}^{C_{\text{in}}}\sum_{i,j=1}^{3} W_{c,i,j}\cdot X_{c,i,j}$

空间滑动（i,j 求和）和**通道混合**（c 求和）**耦合**在一起做。

MobileNet 的洞察

“既然这两件事本质不同，为什么不把它们分开做？”

空间滤波交给 **Depthwise**（每通道独立），通道融合交给 **Pointwise**（1×1 卷积）。



数字例子 ($C_{\text{in}} = 32$, $C_{\text{out}} = 64$)

标准: $32 \times 64 \times 9 = 18432$
深度可分离: $32 \times 9 + 32 \times 64 = 2336$
比例: $\approx \mathbf{1/8}$

通用公式

$$\frac{\text{DS}}{\text{Std}} = \frac{1}{C_{\text{out}}} + \frac{1}{9} \to \mathbf{1/9}$$

一句话记住

标准 = 大锤子一次砸（空间 + 通道耦合）；**深度可分离** = 小锤子砸空间 + 1×1 融通道。**计算量降至** $\sim 1/8$。

应用场景

- **手机相机**：实时人像分割、背景虚化
- **IoT 设备**：智能门铃、智能音箱
- **自动驾驶**：车载轻量检测
- **Android 端 AI**：TFLite 默认模型
- **医疗便携设备**：实时病灶检测

系列演进

- **V1 (2017)**：深度可分离卷积的开山之作
- **V2 (2018)**：**倒残差** + 线性瓶颈
- **V3 (2019)**：**NAS 搜出来**的结构 + SE 模块

来自 Google

作者 Andrew Howard 等，是 TensorFlow Lite 的默认图像模型。

思想传承

深度可分离卷积后来被 **Xception**、**EfficientNet**、**ConvNeXt** 采用，**成为现代 CNN 的标配之一**。

传统做法：只扩一个维度都有天花板

深度 → 梯度难训；宽度 → 通道冗余；分辨率 → 浅层吃不下细节。

复合缩放 (Compound Scaling)

用**单一系数** ϕ 同时控制三个维度：

$$d = \alpha^{\phi}, \quad w = \beta^{\phi}, \quad r = \gamma^{\phi}$$

约束 $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$：$\phi$ 每加 1，计算量 $\times 2$。

两步使用法：缩放的是谁?

第 1 步：NAS 搜出基础模型 **B0**（骨架来自 **MobileNet-V2 的 MBConv + SE 注意力**），定下 $\alpha = 1.2, \beta = 1.1, \gamma = 1.15$

第 2 步：**以 B0 为基准只调** ϕ：$\phi = 1, \ldots, 7 \rightarrow$ B1, ..., B7

B0 → B7 家族

模型	分辨率	参数	Top-1
B0	224	5.3 M	77.3%
B1	240	7.8 M	79.2%
B3	300	12 M	81.7%
B5	456	30 M	83.7%
B7	600	**66 M**	**84.4%**

对标 ResNet

B0 ≈ ResNet-50；B3 ≈ ResNet-152；**B7** > 所有 ResNet

怎么选

手机 → **B0/B1**；服务器 → **B4/B5**；追求极限 → **B7**

一行调用 + 战绩

```
from torchvision.models import efficientnet_b0, efficientnet_b7
```

B7：精度 84.4% > ResNet-152（77.8%），**参数少 8 倍、计算量少 6 倍**。作者 **Tan & Quoc Le**（Google Brain）。

背景：CNN 快被 Transformer 抛弃了

2020-2021 年 ViT / Swin Transformer 横扫视觉任务，**人们开始认为 CNN 过时了**。

ConvNeXt 的反击

FAIR 团队问：**“如果把 Transformer 的所有 trick 移植回 CNN 会怎样？”**

- **大卷积核**（7×7）代替小核
- **LayerNorm** 代替 BatchNorm
- **GELU** 代替 ReLU
- **深度可分离**卷积 + 倒瓶颈结构

结果震惊学界

ConvNeXt 在 ImageNet 上反超 Swin Transformer，证明 CNN 依然有潜力。

启示

ViT 的胜利**不是架构本身**的胜利，而是**现代训练技巧 + 大核设计**的胜利。CNN 用同样的技巧也能赢。

作者

刘壮，谢赛宁等（FAIR + UC Berkeley）。

ViT vs CNN：不是谁取代谁

场景	首选	原因
边缘设备/手机端	**MobileNet / EfficientNet**	计算量小
中等数据集（<10 万张）	**ResNet / ConvNeXt**	CNN 归纳偏置节省数据
大数据集（ImageNet-1K）	两者相当	精度持平
超大数据集（JFT-300M）	**ViT**	Transformer 规模效应强
多模态（图文对齐）	**ViT (CLIP)**	与文本 Transformer 对齐

2020+ 的转折

Vision Transformer（ViT）出现后，CNN 和 Transformer **开始融合**——现代架构常常**前几层 CNN + 后几层 Attention**（混合设计）。**但理解 CNN 的设计思想，依然是理解所有现代视觉模型的基础。**

核心思想

不要从零训练 CNN。下载在 ImageNet 上预训练好的模型，在自己的小数据上微调。

常见做法

1. 用 `torchvision.models` 加载预训练权重
2. **冻结**前面的卷积层（特征提取器）
3. **替换**最后的 FC 层为你的任务（如 10 类）
4. 用小学习率训练几个 epoch

为什么有效

ImageNet 训练出的卷积核学到了**通用视觉特征**（边缘、纹理、形状），**对几乎所有视觉任务都有用**。

小数据集 + 预训练 = 大数据集 + 从零训练。

```python
import torch.nn as nn
from torchvision import models

# 1) 加载预训练 ResNet-18
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)

# 2) 冻结所有参数
for param in model.parameters():
    param.requires_grad = False

# 3) 替换最后的FC层（假设新任务是10类）
num_features = model.fc.in_features    # 512
model.fc = nn.Linear(num_features, 10)  # 只有这层会被训练

# 4) 训练循环和第5讲完全一样
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

效果

在 Cats vs Dogs（1 万张图）上：从零训练 $\sim 80\%$；预训练微调 $\sim 98\%$，5 个 epoch 即可。

五个里程碑

- **AlexNet (2012)**：ReLU + Dropout + GPU，开启时代
- **VGG (2014)**：堆叠 3×3，模块化先驱
- **GoogLeNet (2014)**：Inception + 1×1 + GAP，参数效率
- **ResNet (2015)**：残差连接，让 “深” 成为安全操作
- **现代**：DenseNet / MobileNet / ConvNeXt 等百家争鸣

三大设计原则（带走这三句话）

1. **用小核堆叠**，而不是用大核
2. **用残差连接**让网络可以加深
3. **用 GAP 取代 FC** 减少参数

下一讲（第 8 讲）：循环神经网络与序列建模

- 序列数据 vs 图像数据的本质差异
- RNN 结构与隐状态
- BPTT 与梯度消失/爆炸
- LSTM/GRU 门控机制

课后练习

1. 用 `torchvision.models.resnet18(pretrained=True)` 在 CIFAR-10 上微调，目标精度 $\geq 90\%$
2. 计算 ResNet-18 第一个残差块的参数量并验证
3. 阅读 ResNet 原论文 Figure 1（深网络精度退化），写 200 字感悟
4. 选做：复现 VGG-11 在 Fashion-MNIST 上的训练

谢谢！

问题与讨论

下次课：循环神经网络与序列建模

孟翰　人工智能学院